

Documentation on implementation decisions and assumptions pre programming

Employees

Since we have multiple employees that will likely share most attributes, we will have an abstract employee class from which all specific subtypes can inherit from. The abstract class will also contain a method that will be overridden in each employee class to fit their specific need (delivery boy). This allows the code base to be expandable in the future. Employees will be tied to a branch as each branch has their own set of employees. There was the consideration for a singleton to calculate pay but this would violate SOLID principles and create code smells.

Singleton

The singleton will be an EmployeeManager that holds all employees across all branches. As each employee self-calculates their pay, the singleton will be responsible for hiring/ firing employees and triggering the salary payment, telling the employees to update the money they have based on how many hours they worked.

Pizza

We have the choice between a decorator and a builder pattern. The pizza object will hold a list of ingredients and each ingredient is an object that contains its own price. The price of a pizza can be calculated by iterating through the list, so the calculation stays consistent, allowing maintainability of the code base.

We specifically opt for the builder pattern as the logic for the price calculation stays the same regardless of what ingredient is used. The ingredient lists contains each ingredient used, and as each ingredient contains the individual cost, by iterating we create a slim implementation of the price calculation that does not change. As we do not require a new price calculation logic, implementing the decorator pattern would be unnecessary. By wrapping the base class with multiple decorators, it increases the difficulty for future testing as we would need to traverse the wrapping hierarchy. Furthermore, the price calculation is a simple addition, a recursive approach for a trivial task is computationally unnecessary, and it increases the overhead for maintenance as well as increases the complexity of the code. By using the builder pattern instead of the hinted decorator pattern we keep the code base maintainable, increasing cohesion and reducing coupling. (This deviation from the intended task has been made as the task allows changes if necessary in point 3)

We can also use an abstract factory that receives the pizza object that contains all the ingredients and turns it into a specific sub type (as specified earlier).

The workflow will be that the kitchen aid will first use the abstract factory to create a pizza builder of a specific subtype. The kitchen aid will then continue to use the builder to add the toppings and return the built, raw pizza to the cook, who will then change the state of the pizza

to cooked. This allows clean separation of concerns, preventing smells and promoting SOLID principles.

Additionally, the task hints at possible states of a pizza. While it is possible to implement a state pattern, this would be extremely unnecessary, as the cook will bake the pizza and the state of the pizza will be determined at random based on a weighted distribution. Citing RefactoringGuru: “Applying the pattern can be overkill if a state machine has only a few states or rarely changes.” As the pizza only changes its state twice Raw → Random, the implementation of a state pattern increases the complexity and overhead of the code, making it difficult to maintain in the future. Hence the decision is made to not implement the pattern.

Ingredient

Each ingredient is an object, where the ingredient class is abstract, and cheese, ham and meat are also abstract, extending the base abstract ingredient class. All actual physical ingredient options are then concrete classes. Each ingredient will hold their cost and exp date which will simply be the current date.

Storage

Storage will hold a hashmap of each ingredient object and their count. We can iterate through the dictionary for each key and all their values to check the expiration date. If an ingredient is expired, it is simply removed from the list. Each storage is also bound to a branch with 1:1 multiplicity. A singleton could be implemented to trigger each branch to check their own storage, rather than having the singleton check, creating code envy smell.

Branch

The branch is an abstract class that can create standard branches or specific ones that serve a pizza speciality (as specified above).

Customer

The customer is also an object, it simply creates an order object that gets passed to a branch which then passes it to the coordinator, allowing the start of a pizza creation if possible.

Order

The order is modelled as an object containing what payment type it will have, a list of ordered pizzas and which branch they are contacting. The order is sent to an observer that allows the notification of the specified branch. We determine the branch via an Enum, as each city with a branch will be an Enum.

Documentation on implementation decisions post programming

The codebase is structured into the following packages:

- Employees – contains employee factory and manager singleton
- Ingredients – contains abstract and concrete ingredients, factory for creation
- Order – contains observer for customer and order, manager singleton for routing orders
- Pizza – contains factory to create builders to create pizzas
- Store – Main coordinator of the program

Employees

We have an abstract employee class from which all employees will inherit, the factory pattern is used here to limit the use of new in the codebase. We also implement a singleton to manage all employees. This is the only class that accesses the factory. The singleton is a manager class responsible for the life-cycle of the employee.

Cook has a special busy attribute, allowing him to only work on one pizza per order.

The delivery boy has special attributes allowing him to have his pay calculated differently

Ingredients

We have an abstract ingredient class from which all inherit. Only exceptions are the abstract classes for cheese, ham and meat. These classes act as a container for the specific sub-ingredients. All ingredients are the same, none have any special attributes that deviate from the base abstract ingredient class. We also use a factory whose sole responsibility is to limit the new keyword in the code base and is responsible for ingredient creation. This factory is used by the supplier.

Dough has the special attribute for their thickness, this is then determined by the builder. Each builder will set the respective dough type for their pizza type.

Order

The order package contains the customer and the order related classes. A pizza order is wrapped by multiple classes allowing them to act as pure DTOs.

We first start with an OrderedPizza object, this holds a list of ingredient toppings the customer wants for their pizza.

We then wrap this object with the Order object. The Order object models an actual order, containing the list of the ordered pizzas, and other information such as payment type and the city they are ordering from. This allows us to route the order to the city, preventing orders from Vienna going to branches in Villach.

This is then wrapped again by the OrderProcess object. The purpose of this final wrapping is to implement a subscriber publisher system, the observer pattern. The OrderProcess is the publisher that a customer subscribes to. Any changes made to the OrderProcess will notify the customer, this allows us to alert the customer if the pizza is delayed or on the way, etc. The publisher side of the pattern does not have an interface by design, as to not violate YANGI principle. It does however violate DIP. (Clarification in source code)

Pizza

For the Pizzas there are two patterns implemented, the factory and the builder. The factory creates the concrete builders and the builders then create the pizza. The builders set the dough type based on what builder is being used and allows us to build a pizza with the list of toppings. Originally it was hinted that the decorator pattern was supposed to be used, but that proved unnecessary as the price calculation logic is trivially simple and no new behaviours would be added through different ingredient types. An alternative was selected namely the builder pattern to compensate, as the course heavily focuses on design patterns. The builder pattern aims to solve a telescoping constructor problem. But due to the simple yet elegant implementation of how the Order and Pizzas relate to each other, namely a list of ingredients, the builder pattern loses its purpose and tries to solve a non-existent problem. It now remains as unnecessary code with no benefits.

Nonetheless, the Builder is injected into the kitchen aid to allow specific pizza creation, that fixates the respective dough type and allows the adding of toppings, adjusting the pizzas price. The Builder is injected into the constructor of a Branch object, but can be swapped out for a different one via a setter, complying with the task specs.

Branch

Acts as an orchestrator. Here we inject a storage, the builder into the constructor to ease the testing and mocking for the future, keeping the code base testable as it grows. Each branch holds its own set of employees assigned by the employee manager singleton. It also follows the task specs, allowing us to find free cooks. There are also methods to find other “free” employees, while it does not violate YANGI, the names are slightly misleading but stays consistent with the one for the cook. The cook can only make one pizza per order but can be “overworked” if no more cooks are free. The coordinator checks the order with the storage to see if the ingredients in the storage are sufficient. Should the coordinator find out that the ingredients drop below the desired stock amount, the supplier is then called and the storage is restocked, this then delays the order, notifying the customer. The kitchen aid then walks to the storage to retrieve the ingredients and builds the pizza. The built, raw pizza is then passed to the cook where there is a slight chance of it being burnt, this then also notifies the customer of a delay.

Remarks

While it is true that there are a lack of null checks in the code base, this has been deemed as a minor issue not a critical one. Unlike other projects such as SE2 where an error in production is fatal or when we have unpredictable inputs that could lead to errors, this project is a closed loop environment. The only access to the internals is through the main Application.java where we are hardcoding the values and injecting the objects.

There is also a lack of try catch blocks but the issue would also rely on the Application.java. For example, the exception thrown when we do not have an employee. Implementing a try catch would prevent a crash but the semantics of the program would be broken.

Order arrives → no cook found → try catch triggers → order delayed → hire new cook.
This completely breaks the semantics of a real life pizzeria.

Pre vs Post UML Class Diagram Diffs

All classes received more methods and attributes than the original UML class diagram. This is normal as the first one serves as a guide and an abstraction, meant to capture the core details and core functionality of each class. Reading the spec sheet gives the programmer a vague idea and concept of what the final product will look like and how the UML should be designed to aid during the implementation phase. Hyper specific and concrete classes should be avoided to not impose strict guidelines during the implementation phase. This rule reflects on how the product owner/ customer should not enforce the dev team on how features should be implemented. This violation caused the unnecessary builder in this project, as the factory should have been sufficient. The IDE generated UML diagram provides a detailed breakdown of all methods, fields and properties of a class.

Notable Changes in Structure and Architecture:

Ingredient tree hierarchy stayed largely intact, notable changes were the introduction of suppliers and a factory to create the ingredients. An enum was also created to allow customers to specify which ingredients to use without having to instantiate what ingredients they want, as the creation is the supplier's job.

The factory and builder pattern for the pizza stayed the same, but new enums were created for the pizza to model the state and their type.

The modelling of the ordering system saw the largest deviations from the original UML diagram. While the observer implementation stayed, the interface for the publisher was omitted, as the decision had to be made if DIP or YANGI was more important. In this case, it was decided that the order would not change as all orders can be fundamentally broken down into the same base case and the interface would be redundant and create the YANGI (you aren't gonna need it) code smell. Testing and mocking still remains possible with the current implementation so it was decided to avoid the code smell instead. The original model of the order was a single class, now it is a multi level wrapped object, with each layer serving a different purpose. The inner layer is a DTO that contains a list of toppings, for the pizza creation. This object is wrapped by the actual order that holds a list of ordered pizzas and other information. This object is then wrapped again to allow the implementation of the observer pattern. All inner layers are essentially DTOs.

The employee modelling stayed largely intact with only minor changes but architecturally it is the same. A factory was additionally introduced, which is controlled by the manager singleton. Implementation wise the cook now has more logic than just baking a pizza, he now can be set as busy, preventing them from working on more pizzas. The coordinator also grew in responsibility, as it now not only cross checks the order with the storage ingredients, but it now also is responsible for calling the supplier to restock missing ingredients.

The branch is the class with the most significant changes. It now receives the builder and storage injected and grew in responsibility the most. It now holds a local copy of the employees working in that branch. With the processOrder method exposed and all other methods are helpers.